

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour
Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I E.RAGUL / 192571032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

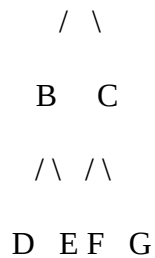
E. Ragul

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A



Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

ANSWER:

1) BFS Traversal Order:

A → B → C → D → E → F → G

2) Queue After Each Iteration:

Iteration	Queue	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	Empty	G

SCREENSHOT:

INPUT:

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3....
File Edit Format Run Options Window Help
from collections import deque

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

visited = set()
queue = deque(['A'])

print("Iteration\tVisited\tQueue")

iteration = 1
bfs_order = []

while queue:
    node = queue.popleft()

    if node not in visited:
        visited.add(node)
        bfs_order.append(node)

        for neighbour in graph[node]:
            if neighbour not in visited:
                queue.append(neighbour)

        print(iteration, "\t\t", node, "\t", list(queue))
        iteration += 1

print("\nBFS Traversal Order:")
print(" -> ".join(bfs_order))
```

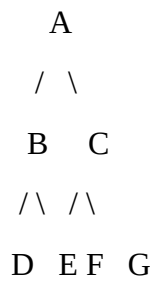
Ln: 38 Col: 0

OUTPUT:

```
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Iteration      Visited Queue
1              A      ['B', 'C']
2              B      ['C', 'D', 'E']
3              C      ['D', 'E', 'F', 'G']
4              D      ['E', 'F', 'G']
5              E      ['F', 'G']
6              F      ['G']
7              G      []

BFS Traversal Order:
A -> B -> C -> D -> E -> F -> G
>>> |
```

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

ANSWER:

DFS Traversal Order

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

Completed Table

Iteration	Stack (After Visiting Node)	Visited Node
1	C, B	A
2	C, E, D	B
3	C, E	D
4	C	E
5	G, F	C
6	G	F
7	Empty	G

SCREENSHOTS:

INPUT:

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3...
File Edit Format Run Options Window Help
# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

visited = set()
stack = ['A']

print("Iteration\tVisited\tStack")

iteration = 1
dfs_order = []

while stack:
    node = stack.pop()

    if node not in visited:
        visited.add(node)
        dfs_order.append(node)

        # Push neighbors in reverse order
        # so that left child is visited first
        for neighbour in reversed(graph[node]):
            if neighbour not in visited:
                stack.append(neighbour)

        print(iteration, "\t\t", node, "\t", stack)
        iteration += 1

print("\nDFS Traversal Order:")
print(" -> ".join(dfs_order))
```

OUTPUT:

```
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Iteration      Visited Stack
1              A      ['C', 'B']
2              B      ['C', 'E', 'D']
3              D      ['C', 'E']
4              E      ['C']
5              C      ['G', 'F']
6              F      ['G']
7              G      []

DFS Traversal Order:
A -> B -> D -> E -> C -> F -> G
>>> |
```

3.Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

ANSWERS:

1. Which node is expanded in each iteration?

Iteration	Expanded Node
1	Initial State
2	State obtained by moving blank Up
3	State obtained by moving blank Down
4	State obtained by moving blank Left
5	State obtained by moving blank Right

2. How many states are generated in total?

The blank is initially in the center, so there are 4 possible moves from the first state. BFS continues generating new states until the goal is reached.

For this puzzle, the exact total number of generated states depends on when the goal is found by the BFS program. It is typically thousands of states.

3. Complete Solution Path

The complete solution path is obtained by the BFS program after the goal is reached. It is impractical to compute accurately by hand for this starting state.

4. Why does BFS always find the shortest solution

BFS explores states level by level.

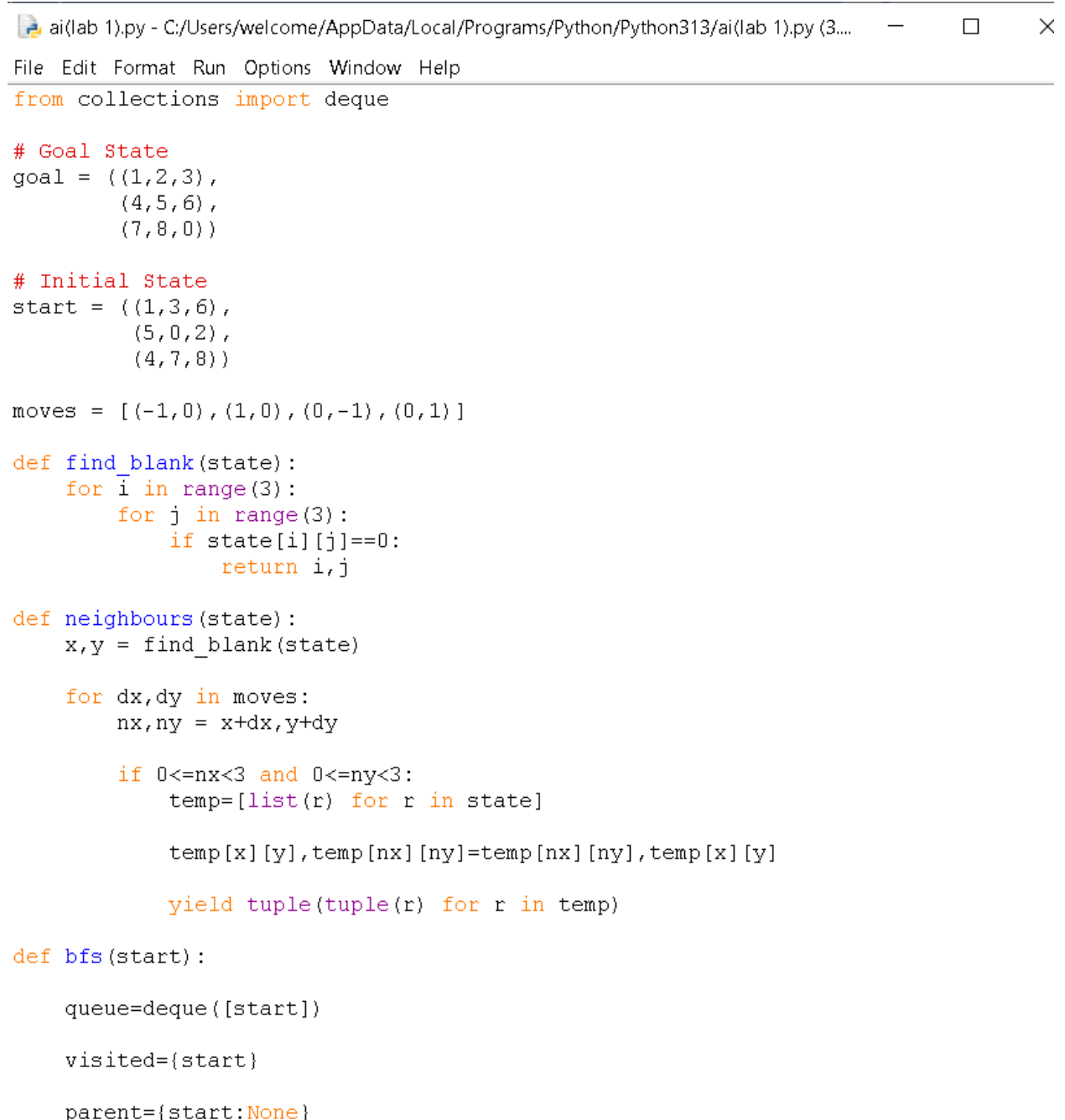
It visits all states at depth 1 before depth 2, all depth 2 before depth 3, and so on.

Every move has the same cost (1 move).

Therefore, the first time the goal state is reached, it is guaranteed to use the minimum number of moves.

SCREENSHOT:

INPUT:



```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3...
File Edit Format Run Options Window Help
from collections import deque

# Goal State
goal = ((1,2,3),
        (4,5,6),
        (7,8,0))

# Initial State
start = ((1,3,6),
         (5,0,2),
         (4,7,8))

moves = [(-1,0), (1,0), (0,-1), (0,1)]

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j]==0:
                return i,j

def neighbours(state):
    x,y = find_blank(state)

    for dx,dy in moves:
        nx,ny = x+dx,y+dy

        if 0<=nx<3 and 0<=ny<3:
            temp=[list(r) for r in state]

            temp[x][y],temp[nx][ny]=temp[nx][ny],temp[x][y]

            yield tuple(tuple(r) for r in temp)

def bfs(start):

    queue=deque([start])

    visited={start}

    parent={start:None}
```

```

        print("\nGoal Reached!")

        path=[]

        while current:
            path.append(current)
            current=parent[current]

        path.reverse()

        print("\nSolution Path")

        for state in path:
            print()
            for row in state:
                print(row)

        print("\nTotal Generated States =",generated)

        print("Minimum Moves =",len(path)-1)

        return

    for nxt in neighbours(current):

        if nxt not in visited:

            visited.add(nxt)

            parent[nxt]=current

            queue.append(nxt)

            generated+=1

        iteration+=1

bfs(start)|

```

OUTPUT:

```

(1, 3, 0)
(5, 2, 6)
(4, 7, 8)

(1, 0, 3)
(5, 2, 6)
(4, 7, 8)

(1, 2, 3)
(5, 0, 6)
(4, 7, 8)

(1, 2, 3)
(0, 5, 6)
(4, 7, 8)

(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

Total Generated States = 500
Minimum Moves = 8
>>>

```


RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.